Departure from Ethereum: Rewriting ZERONYTH from Scratch

What is reusable as-is, what must be redesigned, what must be rewritten

Research report — prepared 2026-11-05. Scope: the blockchain core, the DeFi core, and the wallet core of ZERONYTH, under the owner's confirmed decision to leave the Ethereum ecosystem entirely (no go-ethereum lineage, no Ethereum ecosystem tooling, no public chains) and to rewrite the client from scratch in the Falcon-1024 + ML-DSA-65 (post-quantum) era, with the entire DeFi ecosystem remaining on-chain as the chain's core feature.

Executive summary

The VM design space has converged on a small number of credible shapes. The EVM's synchronous, dynamically-dispatched stack machine delivers unmatched tooling and composability but is sequential, reentrancy-prone, and hard to analyse — and even Ethereum could not afford to modernise its bytecode (EOF was removed from the Fusaka upgrade in April 2025). The credible from-scratch alternatives cluster into: (i) Wasm with declared state footprints (CosmWasm, NEAR, Soroban, Stylus), (ii) resource-typed languages/VMs in the Move mould, and (iii) register/eBPF-style VMs with explicit accounts (Solana SVM, PolkaVM). They differ less in speed than in where trust sits: runtime checks, the type system, or developer discipline. For a DeFi-core chain, the single biggest safety lever the research identifies is **asset semantics enforced by the type system** (Move/Cadence-style resources), and the single biggest functional lever is **keeping a synchronous execution domain** so atomic DeFi composition survives.

The ink!/Polkadot reference must be used with two corrections. First, the widely repeated premise that "ink! v6 is async" is wrong on-chain: ink! cross-contract calls are synchronous, blocking host calls — which is precisely the EVM-like atomic composability DeFi needs. Async exists in ink!'s client/test harness and in Polkadot's cross-chain messaging (XCM), not in the contract model. Second, and decisively: **ink! was formally discontinued on 27 January 2026** by the ink! Alliance after three rejected treasury proposals and a Web3 Foundation/Parity decision not to support it; v6 never left unaudited beta, and its beta binaries are incompatible with the `pallet-revive` Parity stabilised in December 2025. The PolkaVM/RISC-V layer underneath it is healthy (live on Polkadot Hub since 27 January 2026 and specified as JAM's VM), and the architecture — Rust eDSL → typed ABI → host functions → register-based VM — is an excellent reference. But ink! as a product is **not adoptable as a dependency**: adopting this lineage means owning and funding the language stack permanently. Polkadot's central debate (Rust/ink! vs Solidity-first) was settled institutionally in Solidity's favour on network-effect grounds — grounds that, as Section 3 shows, largely do not apply to a private, fixed-team chain.

"Solidity/EVM is uniquely developer-friendly" is partly fact, mostly folklore. Fact: Solidity has the largest contract-specific developer pool, the deepest tooling/audit market, and the fastest time-to-first-contract; that network effect is decisive for a public ecosystem hiring at scale. Folklore: "uniquely" and "easier" as universals. Rust ecosystems onboarded more new crypto developers than Ethereum in 2024 (Solana 7,625 vs Ethereum 6,456); Solidity is easier to start and easier to get catastrophically wrong (access control alone: $953.2M lost in 2024; reentrancy $35.7M in 2024, a decade after The DAO); Rust front-loads difficulty into compilation and repays it by eliminating bug classes by construction. For ZERONYTH's shape — a small, fixed team that hires once and then lives with the safety properties for years — the hiring argument is weak and the safety argument is strong.

Wallets and frontends without Ethereum are a solved problem — elsewhere. Polkadot proves a self-describing chain (runtime metadata + SCALE) can drive fully generic, generated clients (Polkadot-API, subxt) and light clients (smoldot) with zero chain-specific code. ink! proves a contract ABI can be pure metadata JSON consumed by frontends. Solana, Sui, and Aptos prove wallet discovery and signing can be standardised without EIP-1193 (Wallet Standard event discovery, namespaced features, typed rejection). Cartesi proves a deterministic Linux/RISC-V tier can settle heavy compute on-chain — as an optional app-specific tier with optimistic-finality and per-app-node costs, not as the base dApp model. ZERONYTH's wallet core should copy these shapes, with one ZERONYTH-specific imperative Solana teaches in reverse: **post-quantum keys and signatures are huge, so a versioned**

transaction format and pubkey registration/indirection must exist from day one.

Composability: "with async, previous composability is impossible" is fiction as a blanket claim, fact for a specific subset. In a genuinely asynchronous domain (NEAR receipts, XCM) you lose single-transaction atomicity, flash loans as defined, synchronous mid-flow reads, and automatic unwind — and gain explicit state machines, callbacks, escrow patterns, and intents/solvers, which re-create atomicity at the settlement layer. Sui's Programmable Transaction Blocks prove atomicity can also be moved into the transaction format (up to 1,024 commands, all-or-nothing) without a synchronous contract call stack. The real design choice is therefore **where atomicity lives**. For a private chain whose core feature is on-chain DeFi, the cheapest place to keep it is where ZERONYTH's reference stack already has it: synchronous calls inside one execution domain.

Neither Base's nor Stellar's bet transfers directly — but both teach the same lesson. Base rents Ethereum's security and wins on distribution; Stellar added contracts (Soroban) to an established payments L1 and still has only ~$222M DeFi TVL. Ecosystems and asset bases, not execution models, are the scarce resource. ZERONYTH must therefore ship its DeFi legos (token standard, AMM, lending, vaults, oracle) as one coherent, audited, in-house set — and structurally, Stellar's "one synchronous domain first, scale second" is the closer analogue for a DeFi-core private chain. Cross-chain DeFi data underlines the point: Solana is the only non-Ethereum ecosystem with deep production DeFi ($5.54B TVL); Move chains show that safety technology alone does not create DeFi (Sui $469M after a ~$2.1–2.6B peak and the $223M Cetus exploit; Aptos ~$40M).

Bottom line for the mapping (Section 7). ZERONYTH's own post-quantum layer — Falcon-1024 QBFT consensus design, ML-DSA-65 finality, scheme-bound dual-signature transaction design (0x7E/0x7F), and the proven wallet/key-file designs — is **reusable as-is** (as specifications and, where language-compatible, as code). The state model, ABI/encoding, metering, contract standard library, DeFi contracts, client SDKs, and wallet applications must be **redesigned or rewritten** on the new stack; nothing go-ethereum-derived carries over.

---

0. Premises and method

Premises (owner-confirmed, not re-litigated): the departure decision is made; the new client is written from scratch in a language not derived from go-ethereum; Falcon-1024 QBFT consensus and ML-DSA-65 finality proofs are ZERONYTH's own and continue; the whole DeFi ecosystem stays on-chain as a core feature; no public chains are involved. Method: five parallel research tracks (VM/language survey; ink!/PolkaVM deep dive; Solidity-vs-Rust fact-check; non-Ethereum wallet/frontend study; DeFi composability study), each required to cite primary sources (official docs, specifications, repositories, foundation publications, incident post-mortems, market data). Full track notes with inline sources are preserved alongside this report under `research/`. Where the tracks corrected the questions' premises — most importantly on ink! "async" and on ink!'s discontinuation — the corrections are reported as findings, not smoothed over.

---

1. On-chain program platforms: the design space

1.1 The EVM and its compatible chains

The EVM is a stack machine with 256-bit words, per-opcode gas, and synchronous, dynamically dispatched calls (CALL/DELEGATECALL); code and storage live in the same account, and DeFi composability is a direct consequence of that call model [1][2]. A decade of production gives it unmatched tooling, auditors, and liquidity. Modern performance comes from implementations, not the spec: evmone (C++; indirect threading, precomputed gas/stack analysis) and revm (Rust; integrated into Reth, L2s, and zkVM tooling) [4][7][8]. The structural costs are equally documented: sequential execution by default, arithmetic waste, dynamic dispatch enabling reentrancy, and unstructured bytecode that resists static analysis, formal verification, and parallel scheduling.

The attempted fix is the cautionary tale. The EVM Object Format (EOF) — structured containers, deploy-time validation, no dynamic jumps — was removed from the Fusaka/Osaka upgrade in April 2025 citing complexity, timeline risk, and lack of rough consensus; Solidity deleted its EOF backend and evmone 0.18 removed the implementation entirely [3][5][6]. Lesson for ZERONYTH: bytecode-model overhauls carry governance costs even incumbents cannot pay — a from-scratch chain should pick its bytecode model carefully once, because it will live with it.

1.2 Wasm-based systems

Wasm offers structured control flow, typed locals, a formal spec, and near-native speed from mainstream languages (chiefly Rust). The universal catch: **stock Wasm is not consensus-deterministic** (NaN bit-patterns, environment-dependent resource limits), so every chain carves out a deterministic, metered subset — canonicalised floats, banned features, injected metering [25]. The production variants teach distinct lessons:

- **CosmWasm** (Cosmos): contracts are actors that cannot nest calls — a contract returns messages the dispatcher executes — which eliminates classic reentrancy by construction, at the cost of pushing complexity into message/reply sequencing and atomicity handling; upgrades are explicit (`migrate`), not proxies [9].
- **NEAR**: Wasm with asynchronous cross-contract receipts; gas is calibrated to wall-clock (1 Tgas ≈ 1 ms), storage is staked per byte [10][11]. Async simplifies sharding but breaks synchronous DeFi composition (Section 5).
- **Stellar Soroban**: a host/guest split where most work happens in metered host functions under one unified CPU/memory cost model; transactions declare their ledger footprint (read/write set) upfront, enabling fee estimation and potential parallelism; state has TTL/rent via archival [12].
- **Arbitrum Stylus**: a second, co-equal Wasm VM beside the EVM sharing state and ABI, in production since September 2024 — proof that Wasm's value does not require abandoning an account/ABI model; ZERONYTH, leaving anyway, does not need this compromise [13][14].
- **EOS/Antelope**: the earliest Wasm chain; deterministic floating point (SoftFloat) had to be enforced below the contract layer because system contracts used doubles in consensus, and brittle RAM/CPU/NET resource economics undid a fast VM — a fast VM does not rescue weak resource economics [26].

1.3 Move (Aptos / Sui)

Move, from Meta/Diem, is an executable bytecode language built around **linear resource types**: a resource "can never be copied or implicitly discarded, only moved" — enforced by the compiler and an on-chain bytecode verifier on every published module [15][16]. Struct abilities (`copy`, `drop`, `store`, key) make asset duplication and silent loss inexpressible; there is no dynamic dispatch, hence no classic reentrancy; the Move Prover adds specification-level formal verification. The two production forks diverge instructively. **Aptos** keeps account/global storage and adds Block-STM — optimistic parallel execution with multi-versioned data, ~170k TPS on benchmarks, no developer-declared access sets [17]. **Sui** replaces global storage with an object model: transactions list input objects upfront, so non-overlapping transactions execute in parallel and owned-object transfers can bypass full consensus ordering, while shared objects (a DEX pool) still serialise [18]. Limits are real: a small developer pool, a bespoke toolchain to fund, contention re-appearing exactly at shared DeFi state, and safety that stops at the type system — Sui's 2025 Cetus exploit ($223M) was a logic/math bug Move could not prevent (Section 5.4).

1.4 Solana: SVM, sBPF, Sealevel

Solana's VM is register-based, executing programs compiled (via LLVM, mainly from Rust) to sBPF, JIT-compiled to native code. Programs are stateless; everything else is an account owned by exactly one program, and **transactions declare all accounts they will read or write upfront** [19][20]. That declaration is what powers Sealevel: non-overlapping transactions schedule across cores; metering is by compute units rather than per-opcode gas tables. Solana was the general-purpose throughput benchmark for half a decade and remains the deepest non-Ethereum DeFi ecosystem (Section 5.4). Costs are paid by developers: manual account passing into every call, contention collapse on hot accounts, and safety resting on Rust/Anchor conventions rather than VM-enforced asset semantics — account-validation bugs are the dominant exploit class.

1.5 Other approaches worth the design ideas

- **Cairo/Starknet**: programs compile to Sierra, a linear-typed IR that guarantees every program — including failing ones — is provable and gas-tracked before execution as CASM; the fail-safe IR idea (no unbounded/unprovable program reaches the VM) is portable even where STARK proving is not needed [21][22].
- **FuelVM/Sway**: register VM over UTXO; transactions explicitly list the contracts they touch, so execution and validation parallelise; stateless predicates avoid state bloat [23].
- **Cadence/Flow**: resource semantics at the language level — resources exist in exactly one place, move explicitly (`<-`), and cannot fall out of scope — proven at consumer scale (NBA

Top Shot), traded against raw performance and hiring pool [24]. Clarity (Stacks) marks the extreme end: decidable, non-Turing-complete, interpreted on-chain.

1.6 Synthesis: the dimensions that actually decide a from-scratch design

1. **State/access model first, VM second.** Parallelism is decided before any opcode runs: declared access sets (Solana, Fuel, Sui, Soroban footprints) vs optimistic recovery (Block-STM) vs forgoing it (EVM). Shared DeFi state serialises in every design — model pools explicitly.
2. **Determinism is a subsetting discipline, not a format property.** Integers are trivially deterministic; Wasm/eBPF/native compilation all require canonicalisation, feature bans, pinned toolchains, and version-locked execution. Budget for it from day one.
3. **Metering must price the real resources separately** — CPU, memory, I/O, and state growth (rent/TTL/staking models in NEAR, Soroban, Solana are all attempts at the state-growth term).
4. **Asset semantics in the type system is the biggest safety lever.** Move and Cadence eliminate duplication/loss/reentrancy classes by construction; EVM, Wasm/Rust, and Solana leave them to audits and discipline. For a DeFi-core chain this is close to decisive.
5. **Composability model is a product decision** (Section 5): synchronous calls enable atomic multi-protocol DeFi; actor/async models are safer and shard-friendly but fragment atomicity.
6. **Verification follows structure.** A verifying loader (Move), a provable IR (Sierra), or a decidable language enables prover/analysis tooling; unstructured jump bytecode is what EOF tried — and politically failed — to fix.
7. **Upgradeability must be a VM rule, not a proxy pattern**: explicit migration (CosmWasm), compatibility-checked upgrades (Sui), immutable modules (Aptos) all beat delegatecall proxies in auditability.
8. **Tooling surface is the adoption cost.** Reusing Wasm or LLVM/eBPF inherits compilers and debuggers; a bespoke language buys semantics at the price of funding the compiler, SDKs, explorer, indexer, and auditor ecosystem yourself.

---

2. The Polkadot reference in depth: ink!, pallet-revive, and PolkaVM

Because a Rust-based contract stack in the ink! lineage is the leading candidate reference for ZERONYTH's contract layer, this section reports what that stack actually is in 2026 — including two findings that change how it may be used.

2.1 Architecture: a Rust eDSL over host functions

ink! is not a new language but an embedded DSL in ordinary Rust: contracts are normal crates annotated with proc-macros (`#[ink::contract]`, `#[ink(storage)]`, `#[ink(constructor)]`, `#[ink(message)]`), running `no_std`, with explicit storage structs and explicit message mutability [43]. In the native ABI, everything crossing the contract↔host boundary is SCALE-encoded through a pre-allocated static buffer (16 kB in v6) to the pallet's host functions — storage, caller/balance queries, transfers, events, cross-contract calls [43]. The off-chain engine mirrors the same API, so unit tests execute contract logic in-process, and `ink_e2e` runs the compiled binary against a real runtime [45]. v1–v5 compiled to WebAssembly executed by `wasmi` in `pallet-contracts`; v6 compiles to RISC-V (`riscv64emac-unknown-none-polkavm`) executed in PolkaVM inside `pallet-revive` [28].

2.2 What v6 changed — and the async correction

v6 is defined by the migration "`pallet-contracts` + WebAssembly (wasmi) → `pallet-revive` + RISC-V (PolkaVM)", requiring `cargo-contract >= 6` and a chain containing `pallet-revive` [27]. Around it, revive's explicitly Ethereum-first design was imposed on ink! — the v6 alpha notes state the revive team has "Ethereum/Solidity support as the number one priority" [29]. Concretely: `U256` balance contexts; `H160`/`Address` contract addresses with a mandatory 1-to-1 `AccountId ↔ H160` mapping; `H256` hashes; a first-class Solidity ABI mode (`abi = "ink" | "sol" | "all"`, callable from Solidity tooling, at the cost of restricted public types and larger dual-ABI binaries); removal of `call_runtime` and chain extensions in favour of precompile contracts; delegate calls by address rather than code hash [27][29].

Correction — ink! is not async on-chain. Cross-contract calls are synchronous, blocking host calls: `CallBuilder` with `invoke()`/`try_invoke()` returns a value or a handleable error in the same call frame [44], and the official cross-contract-calling documentation shows one contract calling another and using the returned value inline, "just like normal Rust methods" [98]. That is exactly the EVM-like synchronous composability DeFi relies on. Async exists only at two other layers: the client/test side (`#[ink_e2e::test]` functions are `async fn`; deployments are `.submit().await`) [45], and cross-chain messaging via XCM (Section 5.3). Any evaluation premised on "ink! v6's async contract model" is evaluating something that does not exist; the model v6 actually enables is ordinary Rust plus synchronous, type-safe cross-contract calls, with an async Rust harness around it.

2.3 Why PolkaVM: the Wasm → RISC-V move

Parity's stated reasons for abandoning Wasm: nondeterminism risk, and translation cost — Wasm is stack-based while hardware is register-based, making register allocation expensive [30][28]. PolkaVM ingests standard RISC-V ELF binaries from ordinary compilers and translates them offline into a constrained custom bytecode; its lead describes near-`wasmtime` execution with guaranteed O(n), single-pass recompilation hundreds of times faster than Wasm pipelines [32]. The repository's design goals — sandboxed by default, fully deterministic, ≤128 KB baseline memory per instance, a spec simple enough to re-implement an interpreter in under a week — come with the candid caveat that not all are yet true [31]. Deployment is staged: the PolkaVM interpreter runs in-runtime today; a full client JIT is roadmap [30]. PolkaVM is also the execution substrate specified for JAM, Polkadot's planned relay-chain successor — the VM bet is bigger than contracts alone [31].

2.4 Status and roadmap: what is live, what is flux

Live/stable: `pallet-revive` on Kusama (December 2025) and on Polkadot Hub (27 January 2026), where Parity confirms EVM and PVM contracts interoperate natively [37]. By mid-2026 the official docs describe a **dual-VM stack**: REVM (a Rust EVM) runs unmodified Solidity bytecode — "most developers should start with REVM" — while the PVM backend runs `resolc`-compiled Solidity and Rust for performance, with Ethereum JSON-RPC available via an `eth-rpc` adapter [38][39]. Early adoption was tiny (one secondary tally: ~19 contracts a week after Hub launch).

In flux/unfinished: the PolkaVM JIT; PVM tooling maturity ("tooling is still limited" for Rust on PVM [38]); and ink! v6 itself — beta, unaudited, and, per the Alliance, its beta binaries are incompatible with the `pallet-revive` Parity stabilised in December 2025 [33].

2.5 Problems and reality — the discontinuation

ink! v6 never reached production. Three OpenGov funding proposals (Refs. 1760, 1773, 1796) were rejected despite shrinking scope; the Alliance worked unpaid from September to December 2025; on 23 January 2026 the Web3 Foundation and Parity confirmed they saw no strategic case, and on **27 January 2026 ink! was formally discontinued** [33][42]. The Alliance's advice to builders: use `pallet-revive`'s low-level Rust API (explicitly not recommended for a general audience) or rewrite in Solidity [33]. The largest ink! host chain followed: Astar froze new Wasm contract uploads in June 2026, citing the discontinuation and unmaintained code as a growing security liability — its census found only 305 live contracts and 3,101 all-time unique callers [40]. A 2026 status review confirms active maintenance has stopped [41].

This is a **funding/governance failure, not a technical verdict** on Rust contracts — but for planning it is decisive: there is no maintained upstream ink! to adopt.

2.6 The ecosystem's central debate, and what decided it

The Rust side argued from data: Electric Capital figures showing Rust ecosystems onboarding developers ~30% faster, Rust-contract ecosystems at 45.5% vs EVM 40.9% across top-20 ecosystems, ink! as the only PolkaVM-native language at Hub launch, and Rust type safety (one ZK-verifier example claiming ~25% gas and ~45% size advantage over Solidity) [35][34]. Dual-track supporters argued the Hub should onboard via both languages [36]. The Solidity side won on network effects — the existing developer base, audited patterns, OpenZeppelin, MetaMask/Hardhat/Foundry — and on governance's repeated refusal to fund a second common-good language stack; Parity "could not determine sufficient strategic reason to include ink! in the Polkadot Hub roadmap" [33].

Technically, the outcome is not exclusive: both languages share one VM (revive compiles Solidity to PVM; ink! v6 added a Solidity ABI; REVM runs EVM bytecode, with cross-calls between backends) [38]. Institutionally, Solidity is the funded default and ink! lost its funding, maintainership, and main host chain within six months of Hub launch. **The transferable lesson is about ZERONYTH, not Polkadot:** a contract language is permanent

common-good infrastructure. A public ecosystem can let network effects choose it; a private chain must fund and own whichever stack it picks — language, compiler, standard library, audits, tooling — indefinitely. Section 3 shows why, on the merits, a Rust stack remains the right thing to own.

---

3. Fact-check: “Solidity/EVM is uniquely developer-friendly”

The claim (strong folklore form): Solidity is the easy on-ramp to smart contracts, Rust contracts are a niche hardship, so choosing Rust imposes a permanent DX and hiring handicap. Its supporting arguments: gentler learning curve (Solidity looks like JavaScript), a decade of integrated tooling (Remix/Hardhat/Foundry/OpenZeppelin), the largest hiring pool, and overwhelming abundance of examples and forkable DeFi templates. Each contains a real core; the question is whether “uniquely” and “easier, full stop” survive the data.

3.1 Developer population: EVM leads; the “Rust repels newcomers” corollary is false

Electric Capital’s 2024 report (902M commits, 1.7M repositories) counted 23,613 monthly active crypto developers, 6,244 in the Ethereum ecosystem [46]; at VM level the EVM stack stands at ~8,925 developers vs ~2,499 for the SVM (Rust) stack — the EVM network-effect lead is real [47]. But the same report records Solana — Rust contracts — onboarding **7,625 new developers in 2024 vs Ethereum’s 6,456**, the first non-Ethereum ecosystem to beat Ethereum for new developers since 2016, with 83% growth [48]. Across top-20 ecosystems, Rust-contract ecosystems hold 45.5% vs EVM’s 40.9% of full-time developers [49]. Momentum is cyclical rather than lawful (January–September 2025 data puts Ethereum back in front on new developers [50]), and outside crypto Rust’s general pool — 2.27M developers in the prior 12 months by JetBrains’ estimate — dwarfs Solidity’s single-purpose niche [54]. **Reading:** the hiring argument is fact for EVM-specific contract engineers today; it is folklore as a claim about Rust’s overall pool or newcomers’ willingness to learn it.

3.2 Learning curve: faster to first contract vs harder up front

Solidity’s own survey supports the fast start: 36.7% of respondents felt productive in under six months, 17.8% in under a month [51]. Rust’s curve is measurably steeper: the 2024 State of Rust survey (7,310 responses) found only 53% of respondents consider themselves productive, ~31% of non-users cite perceived difficulty, and slow compilation is the top productivity limiter [53]. Two qualifications: “productive in Solidity” often means fast to deployable-looking code, not to safe code — its traps (external-call ordering, delegatecall, proxy initialisation) are typically learned through audits and exploits (§3.3); and post-hump satisfaction with Rust is exceptionally high (Stack Overflow’s most admired language nine years running, 83% in 2024 [53]). Rust’s diagnostics and Clippy function as teaching tooling, not merely friction.

3.3 Safety: what each language prevents by construction — and what neither does

Solidity/EVM’s catastrophic classes are documented in loss data. **Access control** is the #1 category in the OWASP Smart Contract Top 10 (2025): **$953.2M lost in 2024** across 149 incidents totalling $1.42B [55]. **Reentrancy** remains #5 at **$35.7M in 2024** [55], nearly a decade after The DAO’s ~$60M drain forced a hard fork [58], and the July 2023 Curve incident (~$70M gross) located the failure in the compiler — a Vyper reentrancy-lock bug silently disabled protection developers believed they had [57]. **Integer overflow** wrapped by default until Solidity 0.8.0 made arithmetic checked [56], and persists via unchecked blocks and legacy code.

Rust prevents a different, lower layer by construction: ownership/borrowing eliminates use-after-free, double-free, null dereference, data races, and out-of-bounds access in safe code. The strongest field evidence is Google’s Android programme: as new code shifted to memory-safe languages (chiefly Rust), memory-safety vulnerabilities fell from 76% of Android’s total in 2019 (223 flaws) to 24% in 2024 (under 50) [59]. Exhaustive `match`, `Option`/`Result`, and newtypes make illegal states harder to express, and the compiler forces handling of fallible results.

The honest boundary, which auditors repeat: Rust does **not** by construction prevent reentrancy (a contract can still call out and be called back), omitted access-control checks, oracle manipulation, or DeFi logic errors — and OWASP’s top categories are mostly protocol bugs [55]. What a Rust eDSL adds is ergonomic pressure, not magic: explicit typed caller

checks, typed cross-contract call builders, forced `Result` handling — fewer accidental omissions, better static tooling. Conversely, Solidity's ecosystem compensates with mature dynamic defences (Slither, Echidna, Foundry fuzzing/invariants, OpenZeppelin guards) that exist precisely because the language leaves these holes open. Net: "Solidity is easy" conflates easy to write with easy to write safely; the loss record refutes the second. Rust reverses the trade — harder to compile, safer once it does.

3.4 ink! v6 ergonomics, realistically

A developer gets Cargo, rust-analyzer, Clippy, `rustfmt`, `no_std`-compatible crates.io libraries, and testing that is a genuine strength: `#[ink::test]` unit tests against a simulated environment with no node; `#[ink_e2e::test]` end-to-end tests against a real runtime covering cross-contract calls and metering; DRink! for quasi-E2E without a node process [61][62]. `cargo-contract` (≥ v6 for the revive target) builds, deploys, calls, and verifies [27]. Against that: the v5→v6 migration itself was a major breaking change (new binary format, `H160` addressing and account mapping, `no_std` + `no_main`, removed chain extensions, "barebones" tracing/debugging in the beta line) [27]. Solidity is not ergonomically immaculate either: its 2025 developer survey reports "stack too deep" as the #1 recurring pain (47% of respondents; 65% of experts), bytecode-size limits 33%, debugging 33% — ceilings hit hardest by experienced developers — and its tooling argument increasingly rests on one young framework (Foundry, 57% adoption) [52].

3.5 Verdict

- **Fact:** Solidity has the largest contract-specific pool, the most examples/templates, the deepest audit market, and the fastest time-to-first-contract. For a public ecosystem hiring and bootstrapping at scale, that is decisive today.
- **Folklore/outdated:** "uniquely" friendly; "easier" without specifying for what (easier to start, easier to get catastrophically wrong — $953.2M access-control and $35.7M reentrancy losses in 2024 alone); and the implication that Rust contract DX is primitive (Cargo-native unit/E2E testing is at least comparable to Hardhat/Foundry workflows).
- **For ZERONYTH's shape** — a small fixed team building a DeFi core it will operate for years — the hiring-pool argument is weak (hire a handful of engineers once, train them over months) while the safety properties are lived with permanently. The claim is **context-true for ecosystem bootstrapping and folklore as a universal statement**.

3.6 The real constraints of Rust-based contract development (critical review)

1. **Continuity risk (the concrete one):** ink!'s discontinuation (§2.5) means the lineage must be owned/forked, with budget for language, compiler maintenance, and audits — a governance/funding failure, not a technical verdict, but decisive for planning [33][41].
2. **Front-loaded learning curve:** months, not weeks, to onboard Solidity-only hires [53].
3. **Slow compilation** — the Rust survey's top productivity limiter [53]; contract iteration loops are slower than Foundry's.
4. **`no_std` constraints:** on-chain code gets `core` + `alloc` only; many popular crates are unavailable or need `no_std` features [60]; v6 adds `no_main` and new entry-point rules [27].
5. **Binary-size and metering discipline:** size-capped `.polkavm` blobs punish generic-heavy code and large dependency trees (dual-ABI mode inflates size further) [27]; monomorphisation must be managed deliberately.
6. **Macro opacity:** eDSL attribute macros generate dispatch/storage code the developer did not write; trait-bound errors can point into generated code — Rust diagnostics are best-in-class for ordinary code, but eDSL seams are the rough edge.
7. **Thin contract-specific audit/template market:** far fewer experienced auditors and audited DeFi templates than Solidity's; the OpenZeppelin analogue (OpenBrush) is itself affected by the maintenance freeze [41].
8. **Platform churn if tracking someone else's platform:** v5→v6 broke code and storage assumptions within one major version [27]. A from-scratch chain controls its own VM and pins its toolchain — this mitigates the risk, but only if it actually pins and maintains rather than tracks upstream.
9. **Rust does not solve DeFi logic security:** audits, fuzzing, formal verification, and economic review remain mandatory (§3.3).

4. dApp frontends and wallets without Ethereum tools

ZERONYTH cannot reuse the Ethereum client stack even if it wanted to: no Ethereum wallet can produce Falcon-1024 or ML-DSA-65 signatures. This section maps what proven non-Ethereum stacks actually do, because the replacement stack is a solved design problem — four independent ecosystems solved it four times.

4.1 The Polkadot model: the chain describes itself

Polkadot-API (PAPI) requires only runtime metadata ≥ v14 — fetched via `Metadata_metadata*` runtime calls — plus nonce, transaction-validation, and fee runtime APIs; the protocol specification defines that metadata call as returning "all the information necessary to build valid transactions" [63][68]. The metadata enumerates pallets, calls, storage, events, constants, and errors (SCALE-encoded), and clients generate everything from it:

- **PAPI (TypeScript):** `papi add` downloads metadata and generates descriptors; a typed `tx.*`, storage, and events API for that chain follows automatically [64][66].
- **subxt (Rust):** the same metadata generates a strongly-typed Rust API at compile time via a proc-macro over `metadata.scale` [69][70].
- **Polkadot.js extension pattern:** the dApp calls `web3Enable`/`web3Accounts`, receives an injector, and hands the signer to `signAndSend` — keys and the approval popup live in the extension, never in the dApp [72].
- **Light clients:** both PAPI and subxt offer a smoldot provider that connects to a chain, not a trusted RPC node — removing RPC trust, valuable for a private chain with few RPC operators [65][71].

What this buys ZERONYTH: publish one canonical, versioned metadata artifact (calls, storage, events, signed extensions, contract ABIs) and generate every client from it. New transaction types (the PQ 0x7E/0x7F family), runtime upgrades, and fee rules propagate by regenerating descriptors instead of hand-editing every dApp.

4.2 ink!-era frontends: the contract ABI is metadata JSON

`cargo contract build` emits the binary, a metadata `.json` ("which contains the contract's ABI"), and a `.contract` bundle; without the metadata "it's not possible to know what this bytecode refers to" — the JSON is what a dApp UI uses to communicate with the contract [74]. Two generations of tooling share one shape: classically, `ContractPromise(api, metadata, address)` with `contract.query.*` as unsigned dry-runs and `contract.tx.*.signAndSend(...)` for writes [73]; in the PAPI generation, `papi ink add ./contract.json` generates typed descriptors and an Ink SDK provides `getContract`/`getDeployer`, spanning both `pallet-contracts` (v5) and `pallet-revive` (v6+) [67]. Signing never changes shape: build a typed call from metadata → hand a signer payload to the wallet → user approves → broadcast. **For ZERONYTH:** ship every contract with versioned metadata JSON plus a portable type registry, generate TypeScript bindings at build time, and never let frontends hand-roll binary encoding — PQ-sized arguments are exactly where ad-hoc encoding breaks first.

4.3 Solana: the wallet-stack reference to copy

Solana never had MetaMask to lean on, so it standardised discovery and signing:

- **Wallet Standard (chain-agnostic):** wallets dispatch a `register-wallet` event and listen for `app-ready`; dApps do the reverse — "multiple wallets can register and dApps can list them without hardcoding one extension API" [75][76].
- **Wallet Adapter (app side):** provider wrappers plus `useWallet()` expose `publicKey, connected, select, connect, disconnect, signTransaction, signAllTransactions, signMessage, sendTransaction` [77].
- **Phantom-class UX patterns:** connect as origin-scoped account authorisation (not a transaction); one modal listing all discovered wallets; `autoConnect` session restore; distinct `signTransaction` vs `signAndSendTransaction` vs `signMessage` features, each feature-detected per account; a Mobile Wallet Adapter for app-to-app signing [75].
- **Versioned transactions / address lookup tables:** v0 transactions reference on-chain lookup tables by 1-byte index instead of 32-byte addresses, raising the account cap from ~32 to 256 per transaction [78][79]. **For ZERONYTH the lesson is inverted and stronger:** PQ public keys (~1.3–2.7 KB) and signatures (~0.7–3.3 KB depending on scheme) dwarf Ed25519's, so a versioned transaction format from day one, explicit size

budgets, and indirection — address = hash of the PQ public key, key registered on-chain once and thereafter referenced by short address — are not optimisations but survival requirements. ZERONYTH's existing scheme-bound address derivation (address derived from the scheme's own public key, proven in its reference implementation) is exactly the right substrate for this.

4.4 Sui and Aptos: standardisation is cheap if done first

Sui's position: "every self-custodial wallet compatible with Sui implements the Wallet Standard, which means Sui dApp Kit and other apps can discover and connect to them automatically, without any wallet-specific code" — namespaced features (`standard:connect`, `sui:signTransaction`, `sui:signPersonalMessage`, ...), registration via `registerWallet`, discovery via `getWallets()`, and a split between a framework-agnostic core and React bindings with a ready ConnectButton [80][81]. Aptos's AIP-62 does the same (`registerWallet`/`getAptosWallets()`, `aptos:connect`, `aptos:signAndSubmitTransaction`), with `UserResponse<T>` making user rejection a typed outcome rather than an exception [82]. Two unrelated L1s shipped, within ~2 years, a single discovery + feature-namespace standard that made new wallets a non-event for dApps. A chain with initially exactly one wallet can adopt the shape on day one at near-zero cost — and pays heavily if it retrofits it later.

4.5 Cartesi: a Linux VM tier — an architecture option, not a wallet model

Cartesi Rollups split a dApp into on-chain base-layer contracts and an off-chain execution layer: the Cartesi Machine, a deterministic RISC-V emulator (`RV64IMAZicsr_Zifencei`, Sv39 paging) that runs an entire Linux OS, with the machine's whole state mapped to its physical address space for verification [83][84]. All inputs enter through the on-chain `InputBox` and are fed to the machine as `advance_state` requests; outputs are **vouchers** (executable back into L1 contracts, e.g. withdrawals, only after the epoch is finalised), **notices** (provable informational payloads), and **reports** (unproven diagnostics) [85]; frontends read outputs through a GraphQL API [86].

What it suggests for ZERONYTH: a credible pattern for heavy dApps — order books, ML scoring, and notably **PQ signature batch verification/aggregation** — executed in a deterministic Linux/RISC-V sandbox off the validator path, with ordered inputs on-chain and only outputs (state hash + voucher-like executable results) settled on-chain; developers get Rust/Python/C++ and normal Linux libraries instead of a constrained contract language. **Honest limits:** (1) it is an optimistic design — finality waits for epoch claims/dispute resolution rather than QBFT instant finality, so transplanting it means designing ZERONYTH's own proof/challenge rules; (2) it is app-specific — one machine per dApp, no shared composability with native contracts; (3) every app needs reader/validator nodes and an output-serving layer operated; (4) it solves compute, not accounts, signing, or wallet UX. Verdict: an optional execution tier for a few heavy applications, never the base dApp model.

4.6 Synthesis: the from-scratch wallet core

Component	Design decision (mapped to references)
PQ key management	Wallet-only custody (extension/Phantom pattern [72][75]): keys never enter the dApp. Falcon-1024 and ML-DSA-65 secret keys encrypted at rest and scheme-tagged; the backup seed must encode scheme + derivation path, since one seed yields two unrelated PQ keypairs. Hardware-signer abstraction from day one.
Address derivation	Scheme-bound (project spec): address = hash of the scheme's own public key, with scheme/version prefix (SS58-style); a PQ account's address is never derived from an ECDSA key or vice versa. Public key registered on first use; later transactions reference the short address only (§4.3).

Component	Design decision (mapped to references)
Transaction construction	Metadata-driven (PAPI/subxt pattern [63][69]): a versioned chain-metadata artifact generates typed builders for transfers, contract calls, and both PQ transaction types; compact binary encoding; explicit signed fields for nonce, chain ID, and fee — versioned (v0-style prefix) so Falcon, ML-DSA, and future schemes coexist.
Signing flow	Wallet Standard namespacing (§4.3–4.4): zeronyth:signTransaction, zeronyth:signAndSendTransaction, zeronyth:signPersonalMessage, plus standard:connect/disconnect/events. The wallet UI must decode the call from metadata and show human-readable intent (recipient, amount, contract + message, fee) before signing; typed rejection (AIP-62 pattern [82]).
RPC / client layer	Two providers, PAPI-shaped [65]: WebSocket/HTTP JSON-RPC for launch; a light-client provider (smoldot analogue [71]) as the trust-minimised target. Frontends talk to a generated typed API, never raw RPC, except through a documented unsafe escape hatch.
dApp bridge protocol	Wallet Standard discovery verbatim in shape [76]: registerWallet() + app-ready, no window.<brand> globals, no per-wallet code in dApps. Ship (i) a framework-agnostic core, (ii) React bindings + connect modal, (iii) a mobile app-to-app protocol (Solana MWA analogue). One official wallet first — but the standard before a second wallet exists.
Contract frontend	ink! model [74][67]: every contract ships a bundle + metadata JSON; a CLI generates TS bindings; query (dry-run) vs tx (signed) separation everywhere.
Heavy compute (optional)	Cartesi-style deterministic RISC-V sandbox tier for selected dApps (§4.5): on-chain ordered inputs, off-chain Linux execution, on-chain settlement of output hashes and voucher-like outputs. Opt-in per application.

Bottom line: none of this needs Ethereum. Polkadot proves self-describing chains support fully generic generated clients and light clients; ink! proves contract ABIs can be pure metadata; Solana/Sui/Aptos prove wallet discovery and signing standardise cleanly without EIP-1193; Cartesi proves Linux-grade compute can settle on-chain — at costs ZERONYTH should accept only deliberately, per application.

5. What the VM choice means for DeFi composability

5.1 Composability in action: synchronous and atomic

Ethereum’s defining DeFi property is not that contracts can call each other, but that they do so **synchronously, in one transaction, with all-or-nothing revert** — the technical basis

of “money legos” built on shared standards such as ERC-20 and ERC-4626 [88]. Three production patterns depend on it:

- **Flash loans.** Aave’s model is an uncollateralised loan that must be repaid, plus premium, in the same transaction; if not repaid, the transaction reverts “as if it never happened” [87]. No traditional lender can offer this: there is no settlement interval in which to default.
- **Multi-protocol flows in one transaction.** A production liquidator can flash-borrow on Aave, liquidate on Morpho, swap on Uniswap V3, and repay Aave, with repayment checked at the end of the callback [89]. Yield aggregators run the benign inverse: Yearn vaults deposit into Curve pools, stake LP tokens, lend on Aave/Compound, borrow, re-deposit, harvest, and re-compound as composable strategies [117]; ERC-4626 makes the vault share itself composable collateral.
- **Atomic arbitrage and liquidation as system maintenance.** Searchers align prices across DEXes in one revert-or-succeed transaction at almost no inventory risk; liquidations work identically [90]. Atomicity is not a convenience — it is what makes uncollateralised credit, risk-free arbitrage, and single-step leverage/deleverage possible at all.

5.2 The fallout: the same property, weaponised

Composability couples failure domains. The incident record supports four distinct criticisms:

- **Contagion and cascading liquidations.** On Black Thursday (12 March 2020) ETH fell ~50%, Maker vaults went under-collateralised, congested auctions were won with ~0 DAI bids (~62,843 ETH across 1,461 auctions), and Maker was left ~6.65M DAI short, recapitalised via debt auctions [91]. Terra/UST (May 2022) showed cross-system contagion: Anchor held ~70% of UST while Celsius, Voyager, and Three Arrows carried UST/LUNA exposure into a ~$500B market drawdown [92]. Composable, automated liquidation propagates shocks faster than TradFi bank runs, with no lender of last resort.
- **Cross-contract reentrancy surface.** Synchronous callbacks mean a contract can be re-entered before its state settles: The DAO (June 2016, ~3.6M ETH drained, forcing the Ethereum/Ethereum Classic fork) [93]; the July 2023 Curve/Vyper compiler bug (~$70M gross), whose CRV price fall then threatened a large CRV-collateralised borrow on Aave — textbook second-order contagion [57][94]; read-only reentrancy at Conic Finance days earlier, via a Curve callback during `remove_liquidity` [95].
- **Flash-loan-amplified exploits.** Atomicity lends an attacker unlimited temporary capital: Euler Finance lost ~$197M (March 2023) to a flash-loan-funded loop around a missing health check in `donateToReserves` [96]. Flash loans do not create bugs; they make any bug instantly exploitable at full pool scale.
- **MEV enabled by atomicity and ordering.** MEV is value extractable “by including, excluding, and changing the order of transactions in a block” [90]; because an extraction either succeeds atomically or reverts for gas only, it is near-risk-free — one synthesis citing Flashbots/EigenPhi data puts cumulative atomic MEV on Ethereum at ~$1.4B+ since 2020, understated [97]. (A private QBFT chain has levers public Ethereum lacks — a known validator set can adopt ordering policies — but the extraction incentive is created by atomicity itself and must be designed for, not assumed away.)

5.3 Async models — and whether “previous composability is now impossible”

First, the premise correction from Section 2.2 applies with full force here: ink! v6’s on-chain cross-contract calls are synchronous [98][44]. There is no “ink! async model” to lose composability to. The genuinely asynchronous designs in production are cross-domain ones — Polkadot’s XCM is explicitly asynchronous, asymmetric, “fire and forget”, with results requiring a separate message back [99] — and sharded chains like NEAR, whose cross-contract calls “are asynchronous and independent”, execute 1–2 blocks later via promises, whose callback always runs even on failure, and where developers must “manually rollback” in the callback [100][101].

Verdict: fiction as a blanket claim, fact for a specific subset.

What is actually lost inside a genuinely async execution domain: single-transaction atomicity across contracts; flash loans as defined above (no single transaction exists to revert); synchronous price/state reads mid-flow; atomic liquidation-swap-repay; and the guarantee that steps 1–2 unwind automatically when step 3 fails.

What is retained: interface composability (shared token/vault standards, type-safe references), permissionless building on other protocols, parallel independent calls, and multi-step workflows. Contracts still stack — over time, as explicit state machines, instead of over a call stack.

What replaces the lost patterns: (1) **Callbacks and two-phase flows** — escrow/lock in phase 1, settle-or-refund in the callback, with idempotency and timeouts as first-class code. (2) **Intents and solvers** — the user signs a desired outcome, not an execution path; competing solvers fulfil it and settlement is verified on-chain (UniswapX Dutch-auction orders, CoW Protocol batch auctions) [102]. Intents re-create atomicity at the settlement layer while execution stays asynchronous — and mitigate MEV, since orders never sit exposed in a public mempool. (3) **Transaction-level composition** — Sui's different answer: Programmable Transaction Blocks chain up to 1,024 commands whose outputs feed later commands, all succeeding or reverting together; a DeFi aggregator can route across protocols in one PTB "at the displayed price or reverts completely", without deploying a router contract [103][104].

Implication: ZERONYTH's real choice is not "ink! vs composability". It is where atomicity lives: in-VM synchronous calls (EVM; ink!/pallet-revive within one chain), in the transaction format (Sui PTBs), or nowhere — compensated by callbacks and intents (NEAR, XCM). For on-chain DeFi as the core feature, the third option alone is the most expensive; the first is already the reference stack's native behaviour.

5.4 DeFi quality on other chains: Move and Solana vs Ethereum

DefiLlama's chain ranking (4 November 2026) quantifies the maturity gap: Ethereum $45.82B TVL across 2,118 protocols; Base $6.16B; Solana $5.54B / 713 protocols; Sui $468.9M / 161; Stellar $221.9M / 50; Aptos ~$40M [105][107].

- **Solana** is the only non-Ethereum ecosystem here with genuinely deep production DeFi — concentrated and memecoin-heavy: Sanctum ($1.92B), Jupiter ($1.72B), Kamino ($1.31B), Raydium ($1.07B), with $1.91B 24h DEX volume [106]. Jupiter (aggregation) and Kamino (lending/liquidity) deliver strong retail UX and speed. Its flagship failure was economic, not code: Mango Markets lost $114M (October 2022) to oracle-price manipulation of its own collateral token ($67M later returned) [110].
- **Sui** has credible flagship infrastructure (Cetus, DeepBook) and PTBs are a real composability advantage (§5.3) — but depth is thin and fragile: TVL fell from a ~$2.1–2.6B peak to ~$450–470M [108][105], and on 22 May 2025 Cetus lost $223M to an overflow-check bug in a shared math library; validators froze ~$162M, returned after a community vote, with pools refilled partly by a Sui Foundation loan — effective recovery, but a centralisation precedent [109].
- **Aptos**, with the same Move heritage, Block-STM parallelism, and sub-50ms blocks, has not converted technology into liquidity: ~$40M TVL (DefiLlama, November 2026) against a ~$1B peak; its former flagship Thala fell from a $268.5M all-time high to $3.28M [107][111].

Assessment: Move's safety properties reduce some bug classes (notably reentrancy) but demonstrably do not substitute for liquidity, audits, Lindy time, and users (Cetus, Thala). Ethereum's DeFi advantage is less its VM than a decade of accumulated liquidity, tooling, and battle-tested protocol designs. For ZERONYTH — a private chain bootstrapping its own DeFi — this is the operative lesson: **the protocol designs are importable; the liquidity and Lindy time are not, and must be substituted by conservative design, audits, and staged rollout.**

5.5 Architecture note: Base's rollup bet vs Stellar's Soroban bet

Base is an Ethereum L2 on the OP Stack, posting batched data to Ethereum and inheriting its security, with full EVM equivalence [112]; the Superchain vision is a network of L2s sharing "security, communication layers, and a common development stack (the OP Stack itself)" [113]. The bet: rent the trust base, standardise the stack, win on distribution. Costs: sequencer centralisation, liquidity fragmented across L2s, and dependence on another ecosystem's roadmap — underlined by February 2026 reports that Base would migrate off the shared OP Stack to its own unified codebase while still settling to Ethereum [114].

Stellar/Soroban is the opposite bet: keep an established payments L1 (live since 2014) and add general smart contracts — Soroban, Rust-to-Wasm, mainnet March 2024; the 2025 roadmap targets 5,000 TPS via "concurrency, aggressive caching, and ahead-of-time compilation of Soroban smart contracts" [115], and its authorisation framework plus the Stellar Asset Contract bring regulated-asset and fiat-anchor rails Ethereum DeFi lacks natively [116]. Yet Stellar DeFi TVL is only ~$222M [105]: adding contracts to a payments chain did not automatically create a DeFi ecosystem.

Relevance to ZERONYTH: neither bet transfers directly — Base's value is precisely the public Ethereum security and liquidity ZERONYTH is leaving; Stellar's is its public anchor network. Two lessons do transfer. First, **distribution and an existing asset base beat VM novelty**: ecosystems, not execution models, are the scarce resource — ZERONYTH must bootstrap its legos (token standard, AMM, lending, oracle) as one coherent, audited set.

Second, **choose the settlement/atomicity boundary deliberately**: Base accepts async, fragmented cross-L2 composability in exchange for scale; Stellar kept one synchronous L1 execution domain and added contracts to it. For a private chain whose core feature is on-chain DeFi, Stellar's structural choice — one synchronous domain first, scale second — is the closer analogue, even though none of its code or network is reused.

---

6. Implications for the target architecture

Synthesising Sections 1–5 into design commitments for the rewrite:

1. **Execution layer:** a register-based, deterministically-subsetted VM in the PolkaVM mould (evaluate/fork PolkaVM itself — it survived its ecosystem's debate and is JAM's VM [31] — or implement to its design goals), with contracts written in a **thin in-house Rust eDSL** following ink!'s layering (macros → typed ABI → host functions) but owned outright (§2.5–2.6). Do not adopt upstream ink! v6: discontinued, unaudited beta, ABI-incompatible with current `pallet-revive` [33].
2. **Asset semantics as a type-system requirement:** whatever the DSL's surface, token/asset types must be resource-like — non-copyable, non-droppable, moved explicitly — importing Move/Cadence's central lesson into the Rust stack (§1.6) rather than relying on developer discipline.
3. **Atomicity lives in-VM:** synchronous cross-contract calls inside one execution domain (the reference stack's native behaviour [44][98]); PTB-style transaction-level composition [103] is the designated growth path if multi-step user flows outgrow single calls — not an async contract model (§5.3).
4. **Declared access/footprints where cheap:** Soroban/Solana-style declared read/write sets for fee estimation now, parallel scheduling later (§1.2, §1.4); metering that prices CPU, memory, I/O, and state growth separately, with a state-rent/TTL decision made at genesis, not retrofitted.
5. **Metadata-first everything:** one versioned chain metadata artifact generating all clients (§4.1); every contract ships metadata JSON (§4.2); the wallet bridge follows the Wallet Standard shape with `zeronyth:`-namespaced features from day one (§4.3–4.4).
6. **PQ size engineering is a format decision, not an optimisation:** versioned transaction format, scheme-tagged keys, address = hash of the scheme's public key, on-chain pubkey registration with short-address reference thereafter (§4.3).
7. **DeFi ships as one audited legos set** (token standard, AMM, lending, vaults, oracle) with the fallout designed against explicitly: reentrancy discipline in the framework's types, flash-loan-aware health checks, MEV-aware ordering policy options at the QBFT proposer layer, and staged liquidity rollout (§5.2, §5.4–5.5). A Cartesi-style heavy-compute tier is an optional later addition for batch PQ verification and order-book-class workloads (§4.5).

---

7. ZERONYTH subsystem mapping: reusable as-is / redesign / rewrite

7.1 Blockchain core (consensus, finality, execution, networking, state, RPC)

- **Reusable as-is.** The post-quantum layer is ZERONYTH's own and is chain-stack-independent: the Falcon-1024 QBFT consensus design (Falcon-signed QBFT votes, `qbft.falcon1024` genesis semantics, validator key format), the ML-DSA-65 finality-proof design (off-chain attester protocol and its certificate format), and the PQ transaction design — dual native transaction types (Falcon-1024 `0x7E`, ML-DSA-65 `0x7F`), scheme-bound sender derivation (each type derives its sender only from its own scheme's key; no cross-scheme fallback), domain-separated signable messages, and nonce + chainId replay protection. These exist as working, end-to-end-verified specifications from the Besu reference phase (live E2E on test chain 912770: transfers, replay/tamper/wrong-chain rejection, contract creation and calls) and carry over as the new client's normative spec; the underlying Falcon/ML-DSA cryptographic implementations (Bouncy Castle-based Java today) are reusable as reference vectors and test fixtures for the new implementation, and portable where the new stack can bind the same libraries.
- **Redesign.** State and history: the Ethereum Merkle-Patricia trie and Bonsai-style layout are Ethereum-lineage choices, not requirements — the new client needs its own authenticated

state structure chosen for the access model in §6.4. Networking: devp2p is go-ethereum lineage — replace with a libp2p-class or purpose-built authenticated peer layer, re-specified for PQ handshake/signature sizes. The external API: Ethereum JSON-RPC method shapes are replaced by the metadata-driven API of §4.1 (the method semantics — balances, receipts, block queries — carry over as requirements, not as wire format). Fee/metering economics are redesigned per §6.4.

- **Rewrite.** All client code: block production and QBFT state machine implementation, transaction pool (dual-scheme admission, per-scheme gas/size accounting), execution engine host functions, sync, storage engine bindings, and the RPC server — written from scratch in the new stack (Rust, per Sections 2–3). Nothing go-ethereum-derived is carried over, by the owner's decision.

7.2 Contract platform (VM, language, standard library)

- **Reusable as-is.** Design patterns only: ink!'s layering (Rust eDSL → typed ABI → host functions → register VM) [43][44], PolkaVM's design goals (deterministic subsetting, O(n) translation, bounded instances) [31], CosmWasm's explicit-upgrade rule and Soroban's declared footprints as specification inputs (§1.2). No EVM/Solidity code or ABI carries over.
- **Redesign.** The contract ABI and encoding (SCALE-like compact encoding aligned with the chain metadata of §4.1), the metering schedule, contract addressing, and the upgrade/governance rule for deployed contracts (compatibility-checked upgrades in the Sui mould, §1.3, adapted to ZERONYTH governance).
- **Rewrite.** The VM integration, the in-house Rust contract DSL and its compiler pipeline, `cargo`-equivalent contract tooling, and — critically — the **standard contract library** (the OpenZeppelin-equivalent): token, vault, access-control, and math libraries written and audited in-house. Polkadot's experience (§2.5) prices this honestly: it is permanent infrastructure that ZERONYTH must fund itself.

7.3 DeFi core (protocols)

- **Reusable as-is.** The protocol designs: AMM curve mathematics, lending interest-rate and liquidation models, vault strategy logic, and oracle aggregation rules are chain-agnostic specifications and carry over as designs (including the ERC-4626 vault-share concept, re-expressed in the new ABI). Ethereum's production record (§5.1–5.2) is the design test-suite: flash-loan semantics, liquidation flows, and the failure cases (reentrancy classes, health-check omissions, contagion paths) become ZERONYTH's acceptance criteria.
- **Redesign.** The token standard (resource-typed assets per §6.2 — no approve/allowance race patterns), oracle interfaces (signed PQ price attestations fit ZERONYTH's finality layer natively), liquidation and leverage flows tuned to the in-VM atomicity model, and MEV policy (proposer ordering rules available to a permissioned QBFT validator set).
- **Rewrite.** Every DeFi contract — token, AMM, lending, vaults, oracle adapters — in the new DSL, plus off-chain support services (indexers, liquidation keepers, analytics) against the new metadata API. Per §5.4, budget for audits and staged rollout as first-class deliverables: technology does not substitute for Lindy time, so the rollout must manufacture safety margin conservatively.

7.4 Wallet core (keys, signing, dApp bridge, applications)

- **Reusable as-is.** The wallet designs proven in the reference phase: scheme-tagged key files (seed form and expanded secret/public form, interoperable across implementations — the standalone ML-DSA wallet and the Java harness derived identical addresses and cross-verified signatures in live E2E), scheme-bound address derivation, and the dual-scheme signing flows. The JavaScript (noble-based) ML-DSA signing prototype is directly reusable as the seed of the new wallet's crypto layer, since it is ecosystem-independent code.
- **Redesign.** The dApp bridge and approval UX per §4.6: Wallet Standard discovery shape, `zeronyth:`-namespaced signing features, metadata-decoded human-readable approval screens, typed user rejection, and pubkey-registration/indirection for PQ payload sizes.
- **Rewrite.** The wallet applications themselves (browser extension, desktop, mobile) against the new generated client SDKs, the transaction builders (metadata-generated, both PQ types), and the RPC/light-client providers. No EIP-1193/MetaMask compatibility layer is built; Ethereum-wallet compatibility is explicitly out of scope under the departure decision.

7.5 Summary table

Subsystem	Reusable as-is	Redesign	Rewrite
Blockchain core	Falcon-1024 QBFT + ML-DSA-65 finality designs; dual-scheme PQ tx design (0x7E/0x7F, scheme-bound senders); crypto reference vectors	State structure; peer networking; external API (metadata-driven); fee/metering model	All client code (consensus impl., TxPool, execution, sync, storage, RPC) in Rust
Contract platform	ink!/PolkaVM architectural patterns; upgrade/footprint design inputs	ABI/encoding; metering schedule; upgrade rules	VM integration; in-house Rust DSL + tooling; standard contract library
DeFi core	Protocol designs & math; ERC-4626 concept; Ethereum failure-case catalogue as acceptance tests	Token standard (resource-typed); oracle interfaces; liquidation/MEV policy	All DeFi contracts; indexers/keepers/analytics
Wallet core	Key-file formats; scheme-bound address derivation; JS ML-DSA signing prototype	dApp bridge (Wallet Standard shape); approval UX; PQ size indirection	Wallet apps; tx builders; RPC/light-client providers

Principal residual risks. (1) Owning a contract language stack is a permanent cost centre — Polkadot's treasury declined exactly this bill (§2.5); ZERONYTH's smaller scope (one chain, one team, no public ecosystem to please) makes it affordable, but it must be budgeted as forever, not as a project. (2) The auditor market for a bespoke Rust DSL starts at zero; mitigate by keeping the DSL thin over a standard VM and by publishing the standard library early for external review. (3) DeFi bootstrapping without imported liquidity demands conservative parameters and staged rollout (§5.4). None of these risks is a reason to stay: each is a cost of independence that the departure decision has already accepted, and each has a concrete mitigation above.

References

1. Ethereum Whitepaper / EVM execution model — ethereum.org/whitepaper/
2. EVM Opcodes reference — ethereum.org/nb/developers/docs/evm/opcodes/
3. EOFv1 specification — raw.githubusercontent.com/ipsilon/eof/main/spec/eof.md
4. evmone repository (baseline/advanced interpreters, EVMC) — github.com/ethereum/evmone
5. evmone CHANGELOG (EOF removal) — github.com/ethereum/evmone/blob/master/CHANGELOG.md
6. Solidity releases (EOF backend removed) — www.soliditylang.org/blog/category/releases/
7. revm (docs.rs) — docs.rs/crate/revm-context/4.0.0/source/README.md
8. revm book/docs — bluealloy.github.io/revm/docs/revm/index.html
9. CosmWasm contract semantics (actor model) — github.com/CosmWasm/cosmwasm/blob/main/SEMANTICS.md
10. NEAR Gas documentation — docs.near.org/protocol/transactions/gas
11. NEAR nearcore gas architecture — near.github.io/nearcore/architecture/gas/index.html
12. Soroban fees, resource limits and metering — developers.stellar.org/docs/learn/...tals/fees-resource-limits-metering
13. Arbitrum Stylus — optimising Wasm binaries — docs.arbitrum.io/stylus/how-tos/optimizing-binaries
14. Arbitrum Foundation — How Wasm Elevates Performance (Stylus in production) —

blog.arbitrum.foundation/how-wasm-…rformance-without-leaving-the-evm/

15. Blackshear et al., Move: A Language with Programmable Resources — www.readkong.com/page/move-a-langu…ith-programmable-resources-9952091
16. Blackshear et al., Resources: A Safe Language Abstraction for Money — arxiv.org/pdf/2004.05106
17. Aptos Whitepaper (Move, Move Prover, Block-STM) — aptosnetwork-2iegw07k2-aptos-found…whitepaper/aptos-whitepaper_en.pdf
18. The Sui Smart Contracts Platform — github.com/MystenLabs/sui/blob/main/doc/paper/sui.pdf
19. Solana Virtual Machine overview (SVM, Sealevel) — www.alchemy.com/overviews/what-is-the-solana-virtual-machine
20. Solana introduction (Sealevel, BPF programs/accounts) — www.0xversity.com/blockchain/solana-intro/
21. Starknet docs — Understanding Sierra — docs.starknet.io/build/starknet-by-example/advanced/sierra-ir
22. The Cairo Book — Sierra appendix — starknet.io/cairo-book/appendix-09-sierra.html
23. Introducing the FuelVM and Sway — hackmd.io/@l1-programmability/S11fmSCSs
24. Cadence resource tutorial (Flow) — onflow.github.io/docs/cadence/tutorial/resources
25. Deterministic execution / Wasm nondeterminism — dev.cube.exchange/what-is/deterministic-execution
26. EOS/Antelope chain audit (deterministic SoftFloat, RAM economics) — github.com/hilleywyn/snake/blob/HE…its/chains/AUDIT-EOS-RAM-BANCOR.md
27. ink! v5→v6 migration guide — use.ink/docs/v6/faq/migrating-from-ink-5-to-6/
28. Why RISC-V and PolkaVM for Smart Contracts? (use.ink) — use.ink/docs/v6/background/why-riscv-and-polkavm-for-smart-contracts/
29. ink! v6.0.0-alpha release notes — github.com/use-ink/ink/releases/tag/v6.0.0-alpha
30. Smart Contracts on Polkadot (Polkadot Wiki) — wiki.polkadot.com/learn/learn-smart-contracts/
31. PolkaVM repository / design goals — github.com/paritytech/polkavm
32. PolkaVM lead on RISC-V translation & performance (Ethereum Magicians) — ethereum-magicians.org/t/long-term…place-the-evm-with-risc-v/23617/32
33. Discontinuation of ink! (Polkadot Forum, ink! Alliance, 27 Jan 2026) — forum.polkadot.network/t/discontin…rust-smart-contract-language/16849
34. Ink!: Status, Context, and Why It Matters (Polkadot Forum) — forum.polkadot.network/t/ink-statu…-why-it-matters-for-polkadot/16456
35. Rust > Solidity: Data Doesn’t Lie (Polkadot Forum) — forum.polkadot.network/t/rust-solidity-data-doesn-t-lie/15683
36. Should we push Ink or move with Solidity? (Polkadot Forum) — forum.polkadot.network/t/what-do-y…nk-or-move-with-solidity/14031.rss
37. Why Developers Should Deploy Smart Contracts on Polkadot (Parity blog, 27 Jan 2026) — www.parity.io/blog/why-developers-…deploy-smart-contracts-on-polkadot
38. Dual Virtual Machine Stack (Polkadot Developer Docs) — docs.polkadot.com/smart-contracts/for-eth-devs/dual-vm-stack.md
39. Add Smart Contract Functionality / pallet-revive + eth-rpc (Polkadot Developer Docs) — docs.polkadot.com/parachains/custo…/add-smart-contract-functionality/
40. WASM smart-contract sunset on Astar and Shiden (Astar Forum) — forum.astar.network/t/wasm-smart-c…ct-sunset-on-astar-and-shiden/9516
41. ink! Smart Contracts in 2026: Status and What Changed (4IRE) — 4irelabs.com/articles/top-5-challe…building-smart-contracts-with-ink/
42. ink! Alliance Bounty, Ref 1760 (Subsquare) — polkadot.subsquare.io/referenda/1760
43. ink! Architecture (DeepWiki, derived from use-ink/ink) — deepwiki.com/use-ink/ink/1.1-architecture
44. ink! Cross-Contract Calls / CallBuilder API docs — use-ink.github.io/ink/ink_env/call/fn.build_call.html
45. ink! E2E Testing (DeepWiki, derived from use-ink/ink) — deepwiki.com/use-ink/ink/5.2-end-to-end-testing

46. Electric Capital Developer Report 2024 — www.developerreport.com/
47. Blockworks — Solana gained highest share of new devs in 2024 (Electric Capital) — blockworks.com/news/electric-capital-report-solana-developers
48. Cointelegraph — Solana overthrows Ethereum as top ecosystem for new devs — cointelegraph.com/news/solana-flip…ereum-top-ecosystem-new-developers
49. Rust > Solidity full-time developer data (Polkadot Forum, RSS) — forum.polkadot.network/t/rust-solidity-data-doesn-t-lie/15683.rss
50. Electric Capital data, Jan–Sep 2025 (reported) — carigold.com/forum/threads/ethereu…solana-steals-all-the-hype.836749/
51. Solidity Developer Survey 2023 Results — github.com/argotorg/solidity-websi…y-developer-survey-2023-results.md
52. Solidity Developer Survey 2025 Results — www.soliditylang.org/blog/2026/04/…ity-developer-survey-2025-results/
53. 2024 State of Rust Survey Results (Rust Blog) — blog.rust-lang.org/2025/02/13/2024-State-Of-Rust-Survey-results/
54. JetBrains — Rust developer population estimates — blog.jetbrains.com/rust/2025/08/01/rust-vs-java/
55. OWASP Smart Contract Top 10 (2025) — owasp.org/www-project-smart-contract-top-10/ ; data summary — www.insecure.in/blog/owasp-smart-contract-top-10-2025
56. Solidity docs — Checked/Unchecked Arithmetic (0.8.0) — docs.soliditylang.org/en/v0.8.0/control-structures.html?highlight=assert
57. Halborn — Explained: The Vyper Bug Hack (July 2023) — www.halborn.com/blog/post/explained-the-vyper-bug-hack-july-2023
58. Reentrancy / The DAO history — smartcontractshacking.com/attacks/reentrancy
59. Google Security Blog — Eliminating Memory Safety Vulnerabilities at the Source (Android/Rust) — security.googleblog.com/2024/09/el…afety-vulnerabilities-Android.html
60. ink! FAQ — `no_std` / stdlib (use.ink) — use.ink/docs/v5/faq/
61. ink! Testing overview (DeepWiki, derived from use-ink/ink) — deepwiki.com/use-ink/ink/5-testing
62. DRink! quasi-E2E testing for ink! — github.com/use-ink/drink
63. Polkadot-API (PAPI) Requirements — papi.how/requirements/
64. PAPI Getting Started — papi.how/getting-started/
65. PAPI Providers (Smoldot / WebSocket) — papi.how/providers/
66. PAPI Transactions — papi.how/typed/tx/
67. PAPI Ink! SDK — papi.how/sdks/ink-sdk/
68. Polkadot Protocol Specification — Runtime API / Metadata — spec.polkadot.network/chap-runtime-api
69. Polkadot Developer Docs — subxt — docs.polkadot.com/reference/tools/subxt/
70. subxt book — code generation from metadata — docs.rs/subxt/0.38.0/subxt/book/setup/codegen/index.html
71. subxt book — light client (smoldot) — docs.rs/subxt/0.34.0/subxt/book/usage/light_client/index.html
72. Polkadot.js extension usage (web3Enable / signer) — polkadot.js.org/docs/extension/usage/
73. Polkadot.js API-Contract — ContractPromise — polkadot.js.org/docs/api-contract/start/contract.read
74. ink! — Compile Your Contract (metadata / .contract bundle) — use.ink/getting-started/building-your-contract
75. Phantom — Wallet Standard — docs.phantom.com/developer-powertools/wallet-standard
76. Anza Wallet Standard — WALLET.md — github.com/anza-xyz/wallet-standard/blob/HEAD/WALLET.md
77. Solana — How to Connect a Wallet with React (Wallet Adapter) — solana.com/nl/developers/guides/wa…dd-solana-wallet-adapter-to-nextjs
78. Solana — Address Lookup Tables / Versioned Transactions — github.com/bji/solana/blob/HEAD/docs/src/developing/lookup-tables.md
79. Phantom — Send a versioned transaction —

docs.phantom.com/solana/sending-a-transaction-1

80. Sui — Self-Custodial Wallets / dApp Kit — docs.sui.io/onchain-finance/asset-custody/wallets/self-custody
81. Sui — Wallet Standard — docs.sui.io/onchain-finance/asset-custody/wallets/wallet-standard
82. Aptos — Wallet Adapter for Browser Extension Wallets (AIP-62) — aptos.dev/build/sdks/wallet-adapter/browser-extension-wallets
83. Cartesi Rollups Architecture — docs.cartesi.io/cartesi-rollups/1.3/core-concepts/architecture/
84. Cartesi Machine Architecture (RISC-V) — docs.cartesi.io/cartesi-machine/target/architecture/
85. Cartesi Machine Introduction (inputs, vouchers, notices, reports) — docs.cartesi.io/cartesi-machine/
86. Cartesi Rollups GraphQL API Overview — docs.cartesi.io/cartesi-rollups/1.5/rollups-apis/graphql/overview/
87. Aave flash loans (protocol documentation summary) — deepwiki.com/aave/protocol-v2/2.4-flash-loans
88. DeFi money legos / ERC-4626 — boxmining.com/defi-money-legos/
89. Morpho Blue liquidator — external integrations (Aave + Morpho + Uniswap) — deepwiki.com/tsmluky/Morpho-Blue-L...ator-Bot/7.3-external-integrations
90. Ethereum.org — Maximal extractable value (MEV) — ethereum.org/developers/docs/mev/
91. Maker Black Thursday post-mortem — cointelegraph.com/news/maker-debt-...t-mortem-recommends-new-safeguards
92. Terra/UST contagion analysis — github.com/kcolbchain/research/blo...earch/stablecoin-depeg-analysis.md
93. The DAO hack, ten years on (The Block) — www.theblock.co/news/ecosystems/20...-fund-the-dao-hack-10-years-405248
94. Curve Finance / Vyper reentrancy incident explainer — www.bitbase.com/blog/what-is-curve-finance
95. CertiK — Conic Finance incident analysis (read-only reentrancy) — www.certik.com/blog/conic-finance-incident-analysis
96. Euler Finance flash-loan exploit analysis — ponr.org/en/articles/flash-loan-attack-guide.html
97. MEV quantification (Flashbots/EigenPhi data synthesis) — github.com/bdaf-course/bdaf-course...-material/BDAF_MEV_Introduction.md
98. ink! — Cross-contract calling — use.ink/docs/v5/basics/cross-contract-calling/
99. Polkadot Docs — XCM interoperability — docs.polkadot.com/parachains/interoperability/get-started.md
100. NEAR — Cross-contract calls (async receipts) — near-docs.io/smart-contracts/anatomy/crosscontract
101. NEAR — Cross-contract calls tutorial — docs.near.org/smart-contracts/tutorials/cross-contracts/xcc
102. Intents and solvers (UniswapX / CoW) explainer — www.weex.com/news/detail/what-are-...fi-trades-adyp2mcy70m344p7ror1x377
103. Sui for Ethereum developers / Programmable Transaction Blocks — docs.sui.io/getting-started/sui-for-ethereum
104. Sui developer docs (PTBs) — docs.sui.io/develop
105. DefiLlama — Chain ranking / TVL — defillama.com/chains
106. DefiLlama — Solana — defillama.com/chain/solana
107. DefiLlama — Aptos — defillama.com/chain/aptos?volume=true
108. Sui Foundation — Sui vs Aptos — www.sui.io/blog/sui-vs-aptos-move-object-account-model
109. Cetus exploit and recovery analysis — www.jewelswap.io/blog/deepbook-vs-cetus
110. CoinDesk — Mango Markets $114M exploit — www.coindesk.com/business/2022/10/...-himself-returns-most-of-the-money
111. DefiLlama — Thala protocol TVL — defillama.com/protocol/thala
112. Binance Academy — What is Base (OP Stack L2) — academy.binance.com/ro/articles/what-is-base-coinbase-layer-2-network

113. OP Stack / Superchain documentation — docs.optimism.io/op-stack/protocol/getting-started.md
114. Base 2026 stack pivot (report) — blog.web3wagmi.com/base-guide
115. Stellar Development Foundation — 2025 Roadmap — stellar.org/foundation/roadmap
116. Soroban authorisation framework — developers.stellar.org/docs/learn/...contract-development/authorization
117. Yearn vault strategy composition (reference) — gist.github.com/eagleworldpk-bot/8e0f3158cc791dadab7370b75a87339b

Research tracks and full inline-sourced notes: *research/A-vm-languages.md*, *research/B-ink-polkavm.md*, *research/C-solidity-vs-rust.md*, *research/D-wallet-frontend.md*, *research/E-defi-composability.md*. Market data snapshot: 4 November 2026 (DefiLlama).